

Base de Datos Vectoriales

SEMANA 08

Heider Sanchez
hsanchez@utec.edu.pe

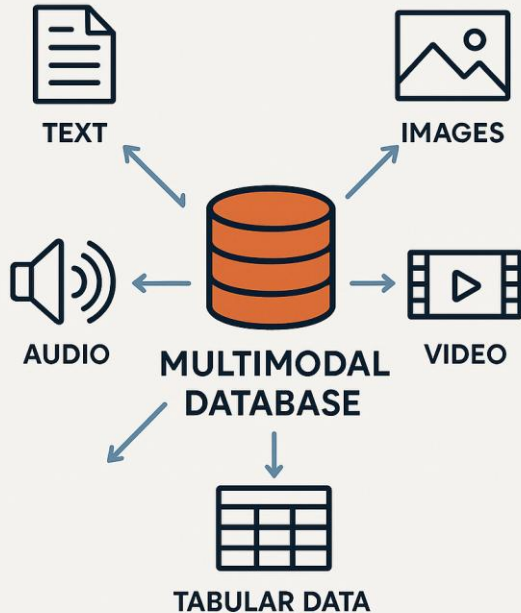
1.



Modelos de Recuperación de Información

Introducción

Tratamiento de Datos No Estructurados



En las aplicaciones modernas, resulta fundamental el procesamiento eficiente de datos no estructurados para permitir consultas **basadas en su contenido**. Esto requiere la integración de técnicas avanzadas de indexación, que **optimicen la velocidad y precisión de las búsquedas**.

```
-- Buscar artículos que contengan la frase "big data y análisis de  
datos" en su contenido
```

```
SELECT id, titulo, contenido  
FROM articulos
```

```
WHERE MATCH(contenido_vector, "big data y análisis de datos");
```

```
-- Buscar rostros similares a una foto de un empleado
```

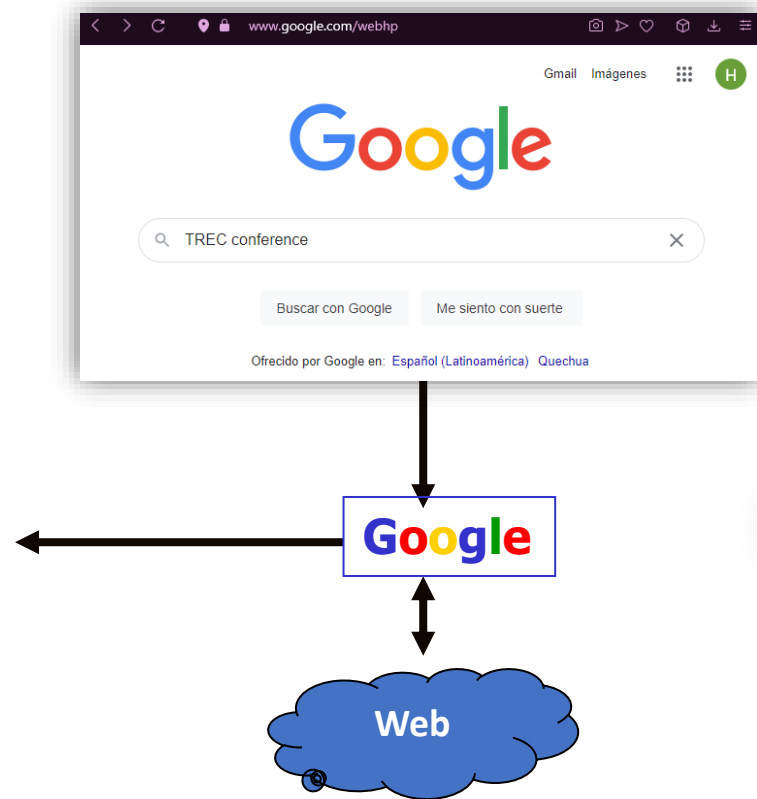
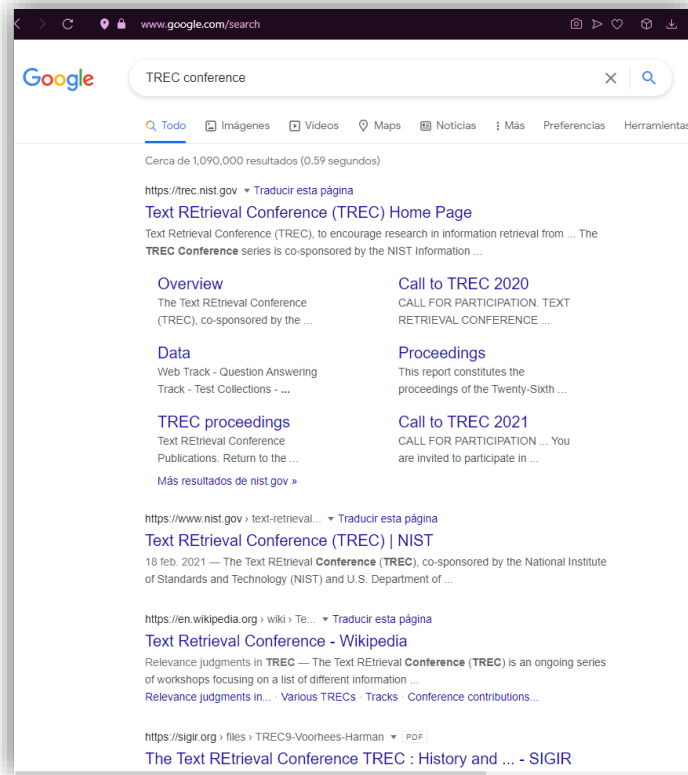
```
SELECT id, nombre, fecha_registro, rostro_path  
FROM fotos_empleados
```

```
WHERE MATCH(rostro_vector, ExtractVector('/path/to/any_photo.jpg'));
```

Ejemplo: Full-Text Search en Repositorio Científico



Ejemplo: Búsquedas en Google



Solución en Base de Datos Relacionales

- Los datos se manejan en tablas de acuerdo al modelo relacional
- El campo textual generalmente es un atributo de tipo **text**
- En las búsquedas se utiliza el operador **like** o **ilike**

Solución en Base de Datos Relacionales

```
Select * from News
where content ilike '%keyword1%'
and content ilike '%keyword2%'
```



Id	Title	Content	Date



Year	Similarity	Author	Article	# of words	Detail
2018	1.0	Casals	brain computer interface with corrupted eeg data a tensor completion approach eeg data tensor completion rehabilitation engineering missing channels performance	# of words: 20067	Abstract
2018	1.0	Castro	inferno inference aware neural optimisation neural network approximate bayesian computation expected variance class/utb01er nuisance parameter	# of words: 5400	Abstract
2018	1.0	Chen	on landscape of lagrangian function and stochastic search for constrained nonconvex optimization >sym/uz713ir/uz2320 oea>oea del/utb01nedcase	# of words: 31248	Abstract

Solución en Base de Datos Relacionales

```
Select * from News  
where content ilike '%keyword1%' and content ilike '%keyword2%'
```

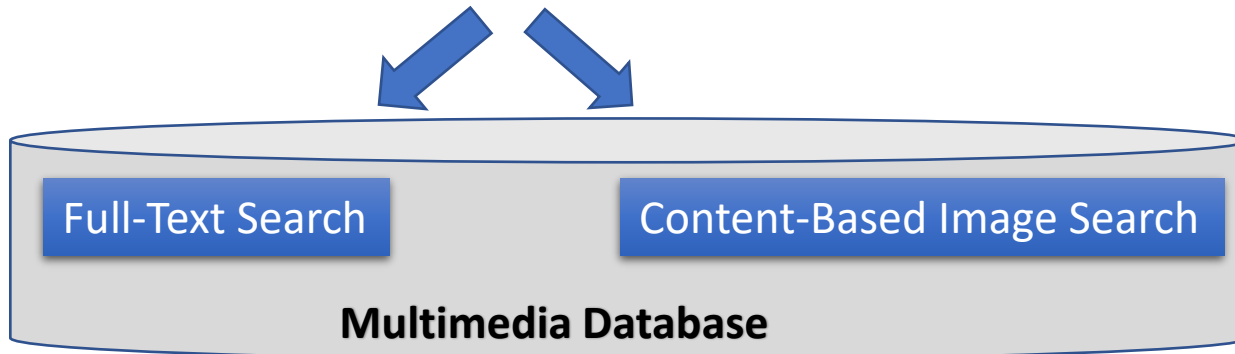
En producción la aplicación no sirve ...

¿Por qué?

- 1. No se considera la relación léxica de palabras en textos escrito por humanos.**
- 2. Demasiados resultados sin ordenar desesperan al usuario.**
- 3. Excesivo tiempo computacional para grandes cantidades de datos.**

Information Retrieval

- **Information Retrieval:** es la ciencia de la búsqueda de información en **documentos digitales**, de naturaleza **no estructurada** que satisfaga una necesidad de información dentro de **grandes colecciones**.



Information Retrieval

- **Information Retrieval:** es la ciencia de la búsqueda de información en **documentos digitales**, de naturaleza **no estructurada** que satisfaga una necesidad de información dentro de **grandes colecciones**.

Full-Text Search: Inverted Index, TF-IDF, Cosine Similarity

Information Retrieval

Artificial Intelligence

Conceptos + Conocimiento
(cuerpo organizado de afirmaciones)

Data Bases

Objetos (entidades)
+
relaciones

Information Retrieval



Information Retrieval

- 1950, Calvin N. Moores: **“La búsqueda de información en un stock de documentos, efectuada a partir de la especificación de un tema”**
- 1983, Salton: **“La recuperación de la información tiene que ver con la representación, almacenamiento, organización y acceso a los elementos de la información.”**
- 1999, Ricardo Baeza: **“La representación y organización debería proveer al usuario un fácil acceso a la información en la que se encuentra interesado. Desafortunadamente dicha caracterización no es un problema sencillo a resolver. ”**

Information Retrieval

- Primeras aplicaciones en bibliotecas (1950s)

ISBN: 0-201-12227-8

Author: Salton, Gerard

Title: Automatic text processing: the transformation, analysis, and retrieval of information by computer

Editor: Addison-Wesley

Date: 1989

Content: <Text>

- Atributos externos y atributos internos (contenido)
- Búsqueda por atributos externos = Búsqueda en BD
- Búsqueda por atributos internos = **IR: búsqueda basado en contenido.**

Information Retrieval

- ¿Que tipo de información **ahora** se necesita recuperar?

- Texto (documentos y/o porsiones de ello)
- XML y otros documentos (semi) estructurados.
- Codigo fuente
- Imágenes
- Audio (efectos de sonido, canciones, etc.)
- Video
- Aplicaciones Web Services

Information Retrieval

Recuperación de Información vs Bases de Datos Relacionales

	Recuperación en BD Relacionales	Recuperación de la información
Acierto	Exacto	Aproximado, El mejor
Inferencia	Algebraica	Inductiva
Modelo	Determinístico	Heurístico, Probabilístico
Lenguaje de consulta	Fuertemente estructurado	No-Estructurado, Natural
Especificación de consulta	Preciso	Imprecisa
Error en la respuesta	Sensible	Insensible

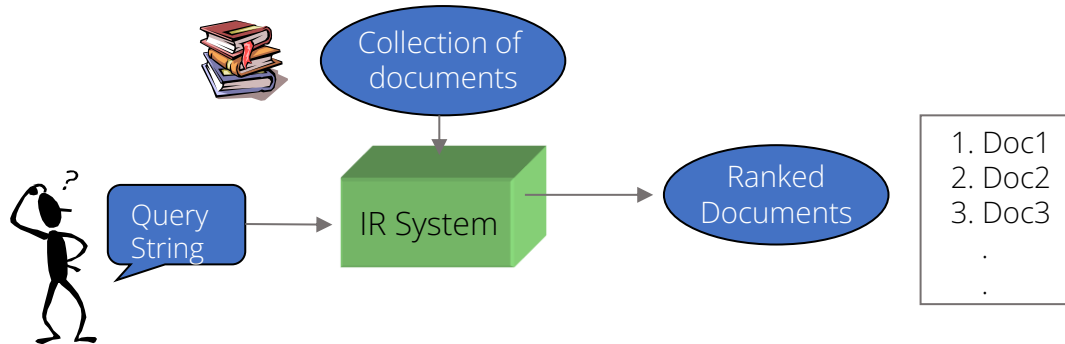
Information Retrieval

- Input:

- Una gran colección de documentos de texto no estructurados.
- Una consulta de usuario expresada como texto.

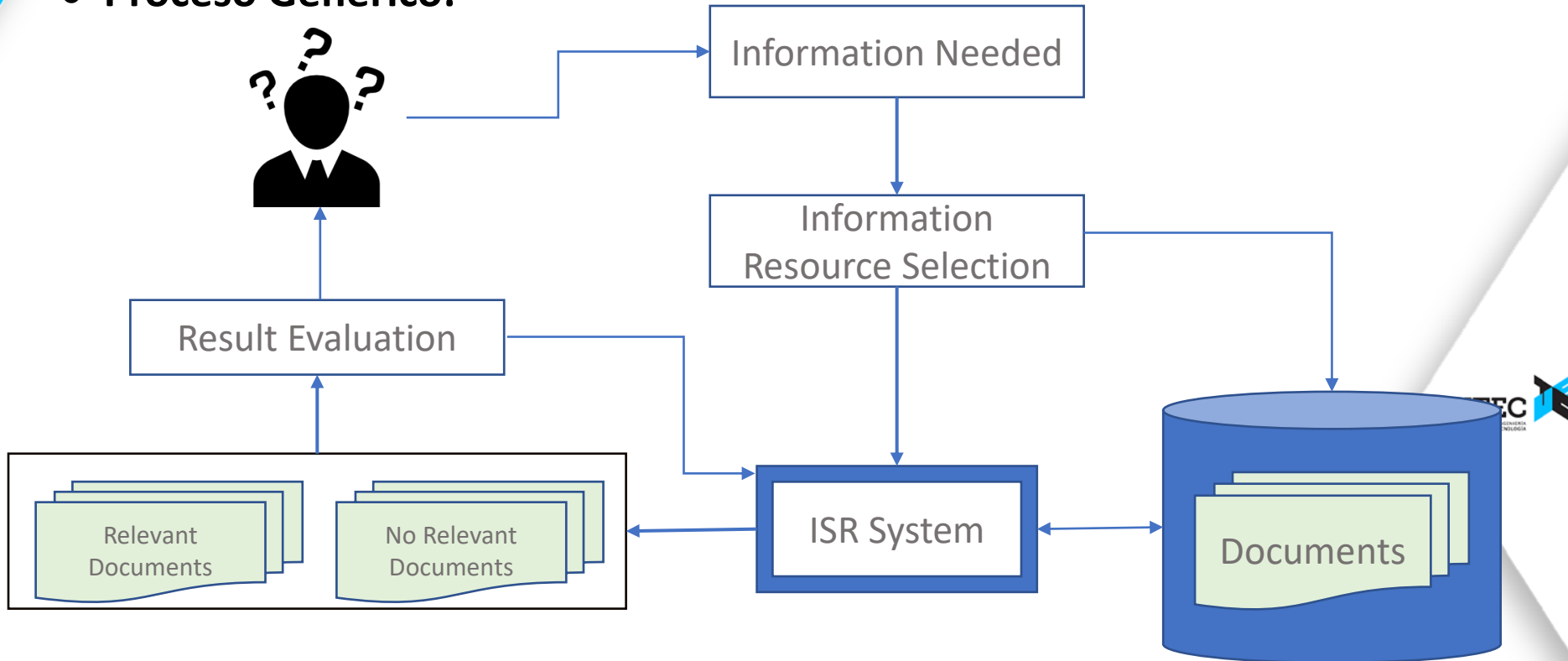
- Output:

- Una lista ordenada de documentos que son relevantes para la consulta.



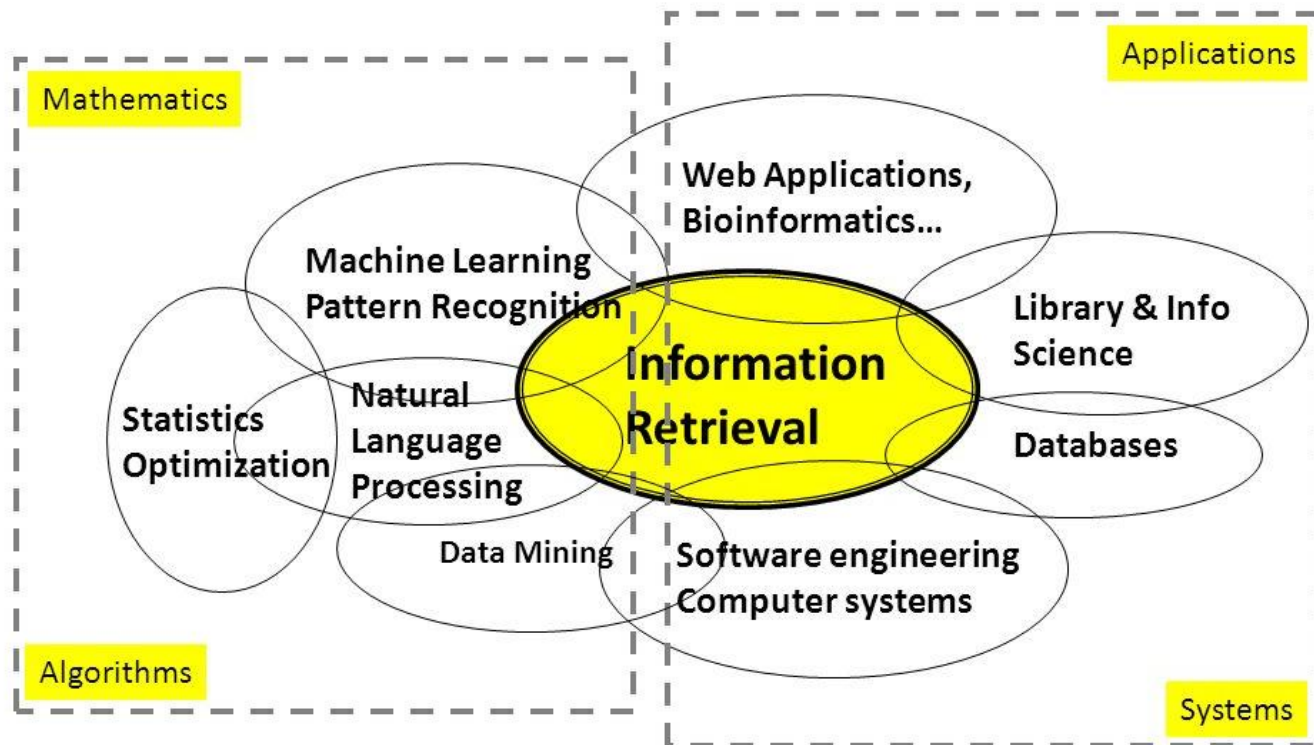
Information Retrieval

- **Proceso Genérico:**



Information Retrieval

- Disciplinas relacionadas



2.



Modelos de Recuperación de Texto

Bag of Words

Solución en Base de Datos Relacionales

```
Select * from News  
where content ilike '%keyword1%' and content ilike '%keyword2%'
```

En producción la aplicación no sirve ...

¿Por qué?

1. No se considera la relación léxica de palabras en textos escrito por humanos.

```
CREATE TABLE articulos (  
  id INT PRIMARY KEY,  
  contenido TEXT  
);
```

```
INSERT INTO articulos (id, contenido) VALUES  
(1, 'Aprender a programar es esencial en el mundo actual.'),  
(2, 'La programación es una habilidad clave para los desarrolladores.'),  
(3, 'Los programadores crean soluciones innovadoras con código.');
```

```
SELECT * FROM articulos  
WHERE contenido LIKE '%programación%' OR LIKE '%desarrollador%';
```



Vectorización del texto con Bag of Words

La técnica Bag of Words es muy utilizada en el Procesamiento del Lenguaje Natural (NLP) para transformar texto en una representación numérica vectorial con el fin de dar soporte a técnicas de Machine Learning e Information Retrieval (que generalmente operan en espacios vectoriales).

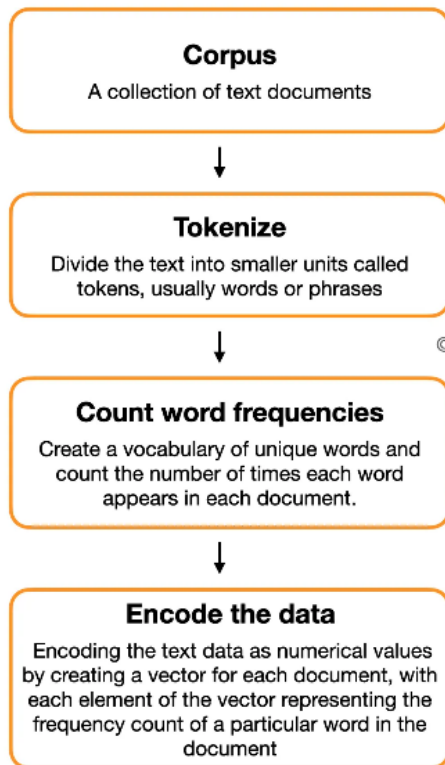
Este vector sirve como input para diferentes tareas :

- Clasificación de textos
- Clustering / Segmentación de textos
- Recuperación de la información (full-text search)
- Sistemas de Recomendación
- Análisis de sentimientos
- Detección de tópicos
- Etc.

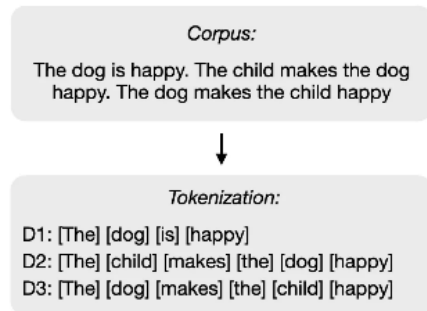
	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

Vectorización del texto con Bag of Words

Bag of Words Model explanation



Example



© AIML.com Research

Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

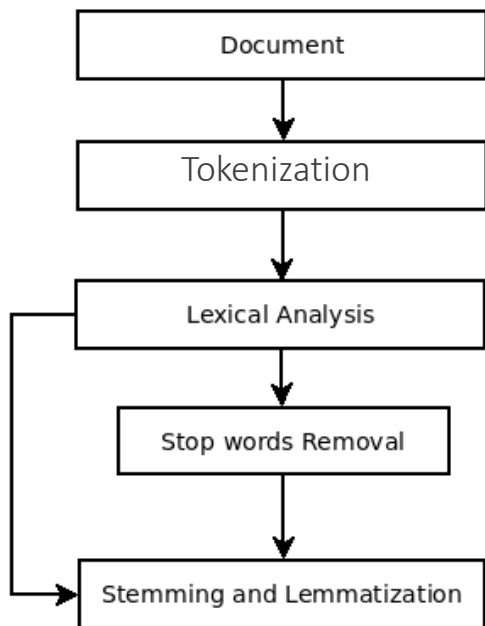
Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

Bag Of Words: Preprocesamiento

Etapas iniciales del procesamiento de texto

- Tokenización
 - Cortar la secuencia de caracteres del texto en tokens de palabras.
- Normalización
 - Mapear el texto y los términos de consulta a la misma forma.
 - ¿Quieres que **O.N.U.** y **ONU** coincidan?
- Reducción (stemming)
 - Podemos hacer coincidir palabras de la misma raíz.
 - **autorizar, autorización**
- Stop words
 - Podemos omitir palabras muy comunes (o no).
 - **de, el, los, uno,**

Bag Of Words: Preprocesamiento



Mis amigas y amigos peruanos
son estudiosos.

amigas

amigos

peruanos

estudiosos

amigo

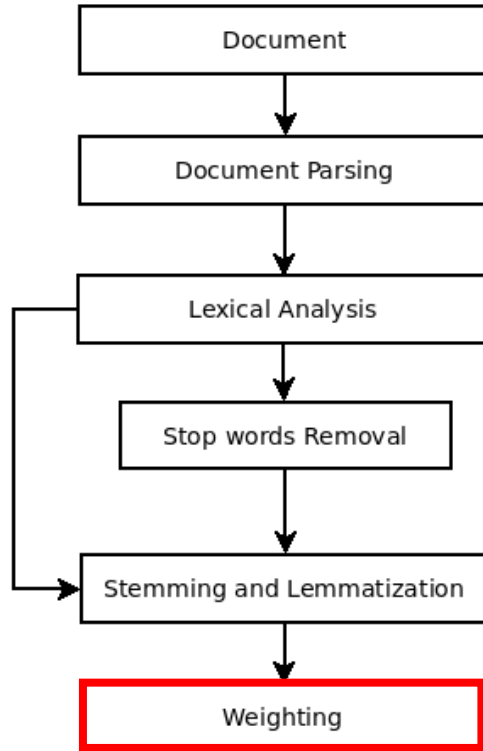
peruano

estudio

Count word frequencies

Create a vocabulary of unique words and count the number of times each word appears in each document.

Bag Of Words: Vectorización



Mis amigas y amigos peruanos
están estudiando.

amigas

amigos

peruanos

estudiando

amigo

peruano

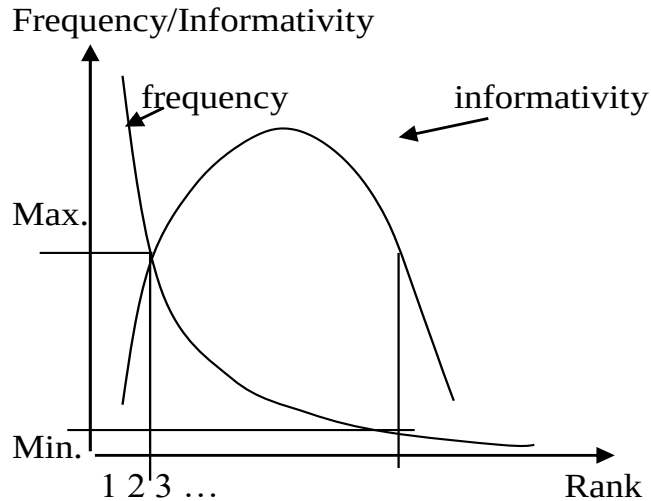
estudiar

[(amigo, w1), (peruano, w2), (estudiar, w3)]

Pesos: incidencia (1/0), conteo, TF-IDF

Bag Of Words: selección de keywords

- ¿Cómo seleccionar los keywords **importantes**?
 - Metodo simple: usando las palabras de frecuencia media



La informatividad de un tipo de palabra es la probabilidad promedio con que ocurrirá dicha palabra, dado cada uno de los contextos en los que puede ocurrir.

$$-\sum_c P(C = c | W = w) \log P(W = w | C = c)$$

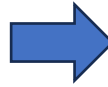
Seyfarth, S. (2014). Word informativity influences acoustic duration: Effects of contextual predictability on lexical representation. *Cognition*, 133(1), 140-155.

Una colección N de documentos
Se extrae todas las palabras que aparecen en los documentos
Se muestra el ranking de las palabras por su frecuencia

Bag Of Words: Ejemplo

Tokenización y Eliminación de Stopwords

```
documentos = [  
    "La casa es grande",  
    "El gato está en la casa",  
    "La casa es bonita y grande",  
    "El sol brilla sobre la casa"  
]
```



```
documentos_procesados = [  
    ["casa", "grande"],  
    ["gato", "casa"],  
    ["casa", "bonita", "grande"],  
    ["sol", "brilla", "casa"]  
]
```



Matriz de Incidencia

	casa	grande	gato	bonita	sol	brilla
Doc 1	1	1	0	0	0	0
Doc 2	1	0	1	0	0	0
Doc 3	1	1	0	1	0	0
Doc 4	1	0	0	0	1	1

Recuperación: Modelo Booleano

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Query = Brutus AND Caesar AND NOT Calpurnia

Answer = $M(\text{Brutus}) \wedge M(\text{Caesar}) \wedge \neg M(\text{Calpurnia})$
 = $110100 \wedge 110111 \wedge 101111$
 = 100100
 $\Rightarrow$ Antony and Cleopatra, Hamlet

110100 $\wedge$
 110111 $\wedge$
 101111
 100100

Bag of Words: PostgreSQL

id integer	content text
1	Un buen día para pasear en moto
2	Todo estaba lindo hasta que tu llegaste
3	No eres lo que logras, eres lo que superas
4	Sube a mi moto mas allá te boto
5	Paseando por las praderas me acorde d...

```
select * from frases
where content like '%pasear%';
```

id integer	content text
1	Un buen día para pasear en moto

```
select * from frases
where to_tsvector('spanish', content) @@
to_tsquery('spanish', 'pasear');
```

id integer	content text
1	Un buen día para pasear en moto
5	Paseando por las praderas me acorde de tí

vector tsvector
'buen':2 'dia':3 'mot':7 'pas':5
'acord':6 'pas':1 'prader':4 'ti':8

Solución en Base de Datos Relacionales

```
Select * from News  
where content ilike '%keyword1%' and content ilike '%keyword2%'
```

En producción la aplicación no sirve ...

¿Por qué?

- ~~1. No se considera la relación léxica de palabras en textos escrito por humanos.~~
2. Demasiados resultados sin ordenar desesperan al usuario.
3. Excesivo tiempo computacional para grandes cantidades de datos.

Laboratorio 8.1



3.



Modelos de Recuperación de Texto

TF-IDF & Similitud de Coseno

Solución en Base de Datos Relacionales

```
Select * from News  
where content ilike '%keyword1%' and content ilike '%keyword2%'
```

En producción la aplicación no sirve ...

¿Por qué?

- ~~1. No se considera la relación léxica de palabras en textos escrito por humanos.~~
- 2. Demasiados resultados sin ordenar desesperan al usuario.**
- 3. Excesivo tiempo computacional para grandes cantidades de datos.**

Problemas con la búsqueda booleana:

- Consulta booleana es bueno para usuarios expertos con una comprensión precisa de sus necesidades y la colección.
 - También es bueno para las aplicaciones que pueden consumir fácilmente miles de resultados para un refinamiento posterior.
- Pero no es bueno para la mayoría de usuarios.
 - La mayoría de usuarios son incapaces de escribir consultas booleanas (o lo son, pero creen que es demasiado trabajo).
 - La mayoría de los usuarios no quieren pasar a través de miles de resultados.
 - Esto es particularmente cierto en la búsqueda web.

Problemas con la búsqueda booleana: “festival o hambruna”

- Las consultas booleanas a menudo dan como resultados muy pocos (= 0) o demasiados (1000).
- Query 1: “*standard AND user AND dlink 650*” → 200,000 results
- Query 2: “*standard AND user AND dlink 650 AND **NOT** card*”: 0 results
- Se requiere mucha habilidad para llegar a una consulta que produzca un número manejable de resultados.
 - **AND da muy pocos**
 - **OR da demasiados**

Ranked Retrieval

- En lugar de un lenguaje de consulta de operadores y expresiones,
 - En Ranked Retrieval, la **consulta** es de **texto libre**, una o más palabras en lenguaje natural.
- En lugar de un conjunto de documentos que satisfacen una expresión de consulta,
 - En Ranked Retrieval, el sistema devuelve un **orden** de documentos de la colección para una consulta dada.
- En principio, hay dos opciones separadas aquí, pero en la práctica, el **ranked retrieval** normalmente se ha asociado con **consultas de texto libre**.

Ranked Retrieval: Scoring

- Scoring es la base de la recuperación por ranking.
- Se desea devolver los documentos en orden para que sean mas útiles al usuario.
- ¿Cómo podemos ranquear los documentos de la colección con respecto a una consulta?
- **Asignando un score, entre $[0, 1]$, a cada documento.**
- Este score mide que tan bien “coinciden” el documento y la consulta.

Ranked Retrieval: Scoring

- Necesitamos una forma de asignar un score al par
 <consulta , documento>
- Empecemos con un término de la consulta
 - Si el término de la consulta no ocurre en el documento, el score debería ser 0.
 - Cuanto más frecuente sea el término en el documento, mayor será el score (debería serlo).
- Vamos a ver una serie de alternativas para esto.

Term Frequency TF

UTEC

Ranked Retrieval: Term-document count matrices

- Considere la cantidad de ocurrencias de un término en un documento:
 - Cada documento es un **vector de conteo** $\mathbb{N}^v$.

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Ranked Retrieval: Term frequency TF

- La frecuencia del término $tf_{t,d}$ es definido como el número de veces que ocurre el término t en el documento d .
- → Queremos utilizar tf para calcular el score de coincidencia entre documento y consulta. ¿Pero cómo?
- La frecuencia del término en bruto no es lo que queremos:
 - Un documento con 10 apariciones del término sería más relevante que un documento con una sola ocurrencia del término.
 - Pero no es 10 veces más relevante.
- **La relevancia no aumenta proporcionalmente con la frecuencia del término.**

Tener en cuenta: frecuencia = conteo en IR

Ranked Retrieval: Log-frequency weighting

- El log-frequency weight de un término t en d es:

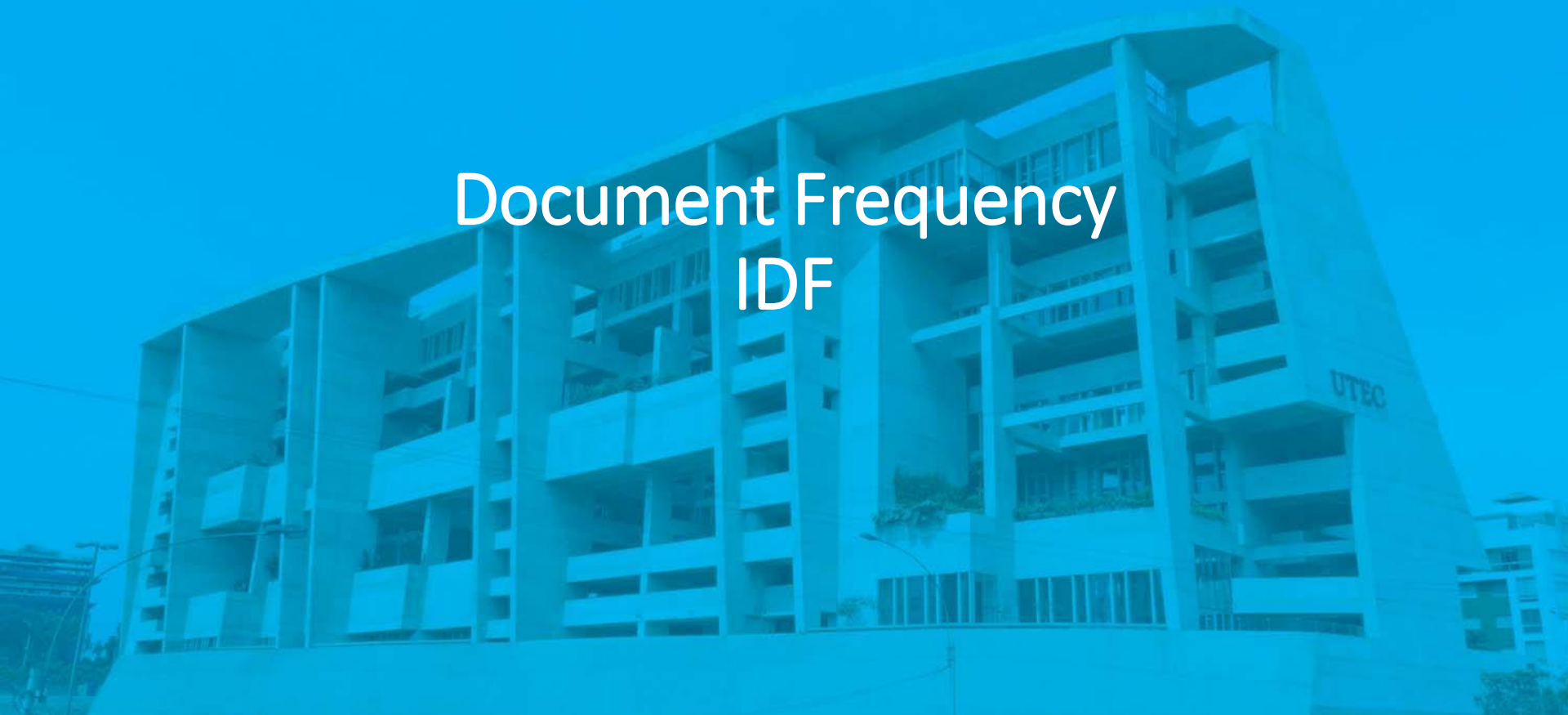
$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d}, & \text{if } \text{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 1000 \rightarrow 4$, etc.
- El score para el par documento-consulta: suma de los términos t que coinciden en ambos q y d :

$$\text{Score}(q, d) = \sum_{t \in q \cap d} (1 + \log \text{tf}_{t,d})$$

- El score es 0 si ninguno de los términos de la consulta está presente en el documento.

Document Frequency IDF



Ranked Retrieval: Document frequency

- Los **términos raros** tienden a ser más informativos que los términos más frecuentes
 - Recordar los stop words
 - Considere un término en la consulta que sea raro en la colección
 - Por ejemplo, *aracnocéntrico*
 - Un documento que contiene este término es muy probable que sea relevante para la consulta *aracnocéntrico*.
- Queremos un peso elevado para términos raros como *aracnocéntrico*.
- Los **términos frecuentes** son menos informativos que los términos raros.

Ranked Retrieval: Document frequency

Los **términos frecuentes** son menos informativos que los términos raros.

Ejemplo: en una colección de documentos técnicos tenemos la siguiente frecuencia:

“para” → 50 veces,

“computación” → 25 veces,

“indexación” → 5 veces,

“hardware” → 1 vez

Ranked Retrieval: Document frequency

- Ahora, considere un término de consulta que sea frecuente en la colección (por ejemplo: alto, aumentar, línea).
 - Es más probable que un documento que contenga dicho término sea más relevante que un documento que no lo tenga.
 - Pero no es un indicador seguro de relevancia.
- Para términos frecuentes, queremos pesos altos y positivos, para palabras como: alto, aumentar y línea.
- Pero a su vez, sus pesos deben ser más bajos que para los términos raros.
- Usaremos la frecuencia de documentos (df).

Ranked Retrieval: IDF weight

- df_t es la frecuencia de documento de t : número de documentos que contienen a t
 - df_t es una medida inversa de la informatividad de t
 - $df_t \leq N$
- Definimos idf (frecuencia de documento inverso) de t mediante:
 - Usamos $\log(N/df_t)$ en lugar de N/df_t para "amortiguar" el efecto de idf .

$$idf_t = \log_{10} (N/df_t)$$

La base del log es irrelevante

Ranked Retrieval: IDF weight

- Ejemplo, suponer $N = 1$ millón

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

$$idf_t = \log_{10} (N/df_t)$$

Hay un solo valor idf para cada término t en una colección.



TF-IDF

UTEC

Ranked Retrieval: TF-IDF weighting

- El peso TF-IDF de un término es el producto de sus pesos TF e IDF.
- Es el mejor esquema de ponderación conocido en recuperación de información:

$$w_{t,d} = \log_{10}(1 + \text{tf}_{t,d}) \times \log_{10}(N/\text{df}_t)$$
$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

- **Aumenta con el número de ocurrencias dentro de un documento.**
- **Aumenta con la rareza del término en la colección.**

Ranked Retrieval: TF-IDF weighting

- $w_{t,d} = \log_{10}(1 + \text{tf}_{t,d}) \times \log_{10}(N/\text{df}_t)$
- $w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$
 - $\text{tf}_{t,d}$: Es la **frecuencia** del término, es decir es el número de veces que ocurre el término t en el documento d .
 - df_t : Es la frecuencia de documento del termino t , es decir es el número de documentos que contienen a t
 - N/df_t : Es la inversa de la frecuencia documental, mide la **rareza** de un termino en la colección.
 - Usamos log para "amortiguar" el efecto de los valores grandes

Ejemplo de pesos TF-IDF

- Documentos:
 - D_1 : “Cargamento de oro dañado por el fuego”
 - D_2 : “La entrega de la plata llegó en el camión color plata”
 - D_3 : “El cargamento de oro llegó en un camión”
- Consulta: “oro plata camión”
- Vocabulario: llegó, dañado, entrega, fuego, oro, plata, cargamento, camión, color.

Ejemplo de pesos TF-IDF

- Las representaciones vectoriales de D_1 , D_2 y D_3 son, respectivamente, $\vec{W}_1$, $\vec{W}_2$ y $\vec{W}_3$ (las tres últimas columnas de la tabla)

Término t	$tf_{t,1}$	$tf_{t,2}$	$tf_{t,3}$	df_t	N/df_t	idf_t	$\vec{W}_1$	$\vec{W}_2$	$\vec{W}_3$
llegó	0	1	1	2	1.5	0.1761	0	0.1761	0.1761
dañado	1	0	0	1	3	0.4771	0.4771	0	0
entrega	0	1	0	1	3	0.4771	0	0.4771	0
fuego	1	0	0	1	3	0.4771	0.4771	0	0
oro	1	0	1	2	1.5	0.1761	0.1761	0	0.1761
plata	0	2	0	1	3	0.4771	0	0.9542	0
cargamento	1	0	1	2	1.5	0.1761	0.1761	0	0.1761
camión	0	1	1	2	1.5	0.1761	0	0.1761	0.1761
color	0	1	0	1	3	0.4771	0	0.4771	0

Ranked Retrieval: TF-IDF weighting

- El score para un documento dado una consulta sería:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

- Hay muchas variantes:
 - Cómo se calcula “tf” (con / sin log)
 - Si los términos en la consulta también están ponderados...

Similitud de Coseno



Ranked Retrieval: Vector Space

- Vector space = all the terms encountered

$$[t_1, t_2, t_3, \dots, t_n]$$

- Document

$$D = [a_1, a_2, a_3, \dots, a_n]$$

a_i = weight of t_i in D

- Query

$$Q = [b_1, b_2, b_3, \dots, b_n]$$

b_i = weight of t_i in Q

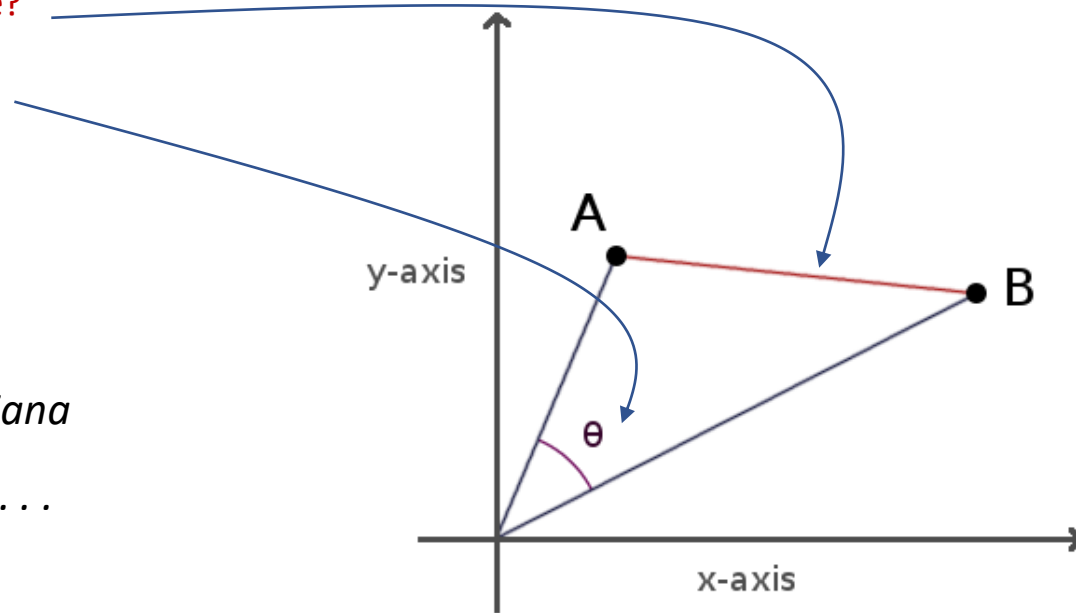
- $\text{Score}(D, Q) = \text{Sim}(D, Q)$

Ranked Retrieval: Proximidad

- Formalización del espacio vectorial de proximidad.

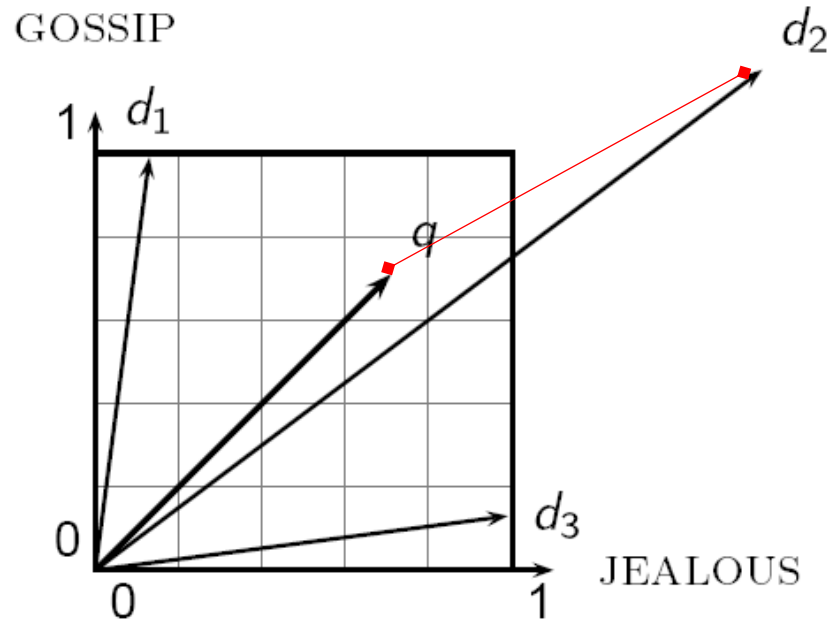
- Euclidean distance?
- Angle distance?

- *Distancia Euclidiana es una mala idea. . .*



Ranked Retrieval: Proximidad

La distancia Eucladiana entre q y d_2 es grande aunque la distribución de términos en la consulta q y la distribución de términos en el documento d_2 son muy similares.



La distancia Eucladiana es grande para vectores de diferentes longitudes.

Ranked Retrieval: Proximidad

- **Usando el ángulo en lugar de la distancia**

- Experimento mental: tome un documento d y adjúntelo a sí mismo. Llama a este documento d' .
- “Semánticamente” d y d' tienen el mismo contenido.
- La distancia euclidiana entre los dos documentos puede ser bastante grande.
- El ángulo entre los dos documentos es 0, el cual corresponde a la similitud máxima.
- **Key idea: rankear los documentos según el ángulo que forman con la consulta.**

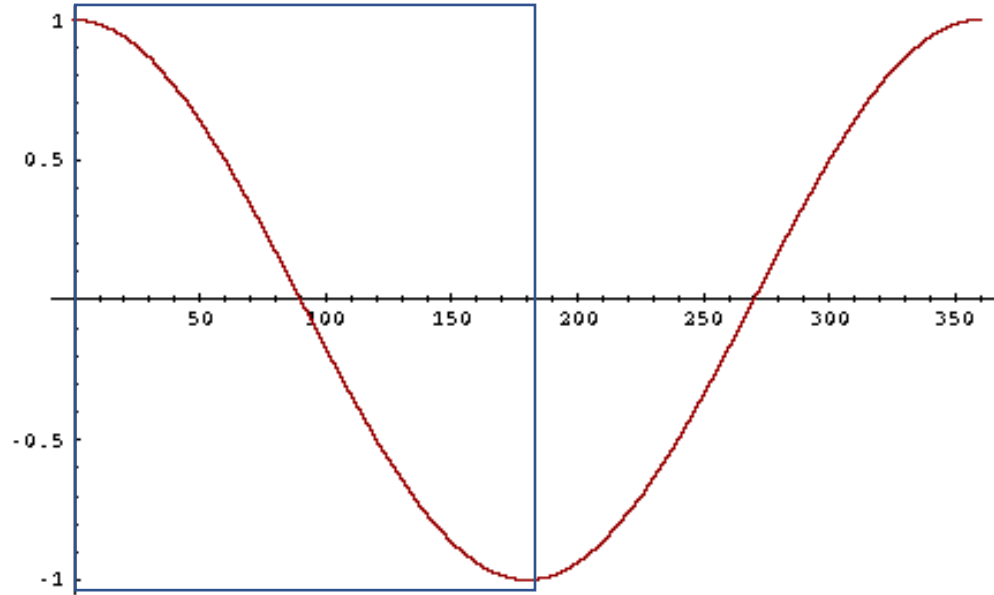
Ranked Retrieval: Proximidad

- **De ángulos a cosenos**

- Las dos nociones siguientes son equivalentes:
 - Ordenar los documentos en orden decreciente del ángulo entre la consulta y el documento.
 - Ordenar los documentos en orden creciente de coseno (consulta y documento)
- El coseno es una función monótonamente decreciente para el intervalo $[0^\circ, 180^\circ]$

Ranked Retrieval: Proximidad

- De ángulos a cosenos



- ¿Cómo --y por qué-- debemos calcular los cosenos?

Ranked Retrieval: cosine(query,document)

Dot product

Unit vectors

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \bullet \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\vec{q}}{|\vec{q}|} \bullet \frac{\vec{d}}{|\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

q_i is the tf-idf weight of term i in the query

d_i is the tf-idf weight of term i in the document

$\cos(\vec{q}, \vec{d})$ is the cosine similarity of $\vec{q}$ and $\vec{d}$... or,
equivalently, the cosine of the angle between $\vec{q}$ and $\vec{d}$.

Ranked Retrieval: $\text{cosine}(\text{query}, \text{document})$

- **Normalizar la longitud:**

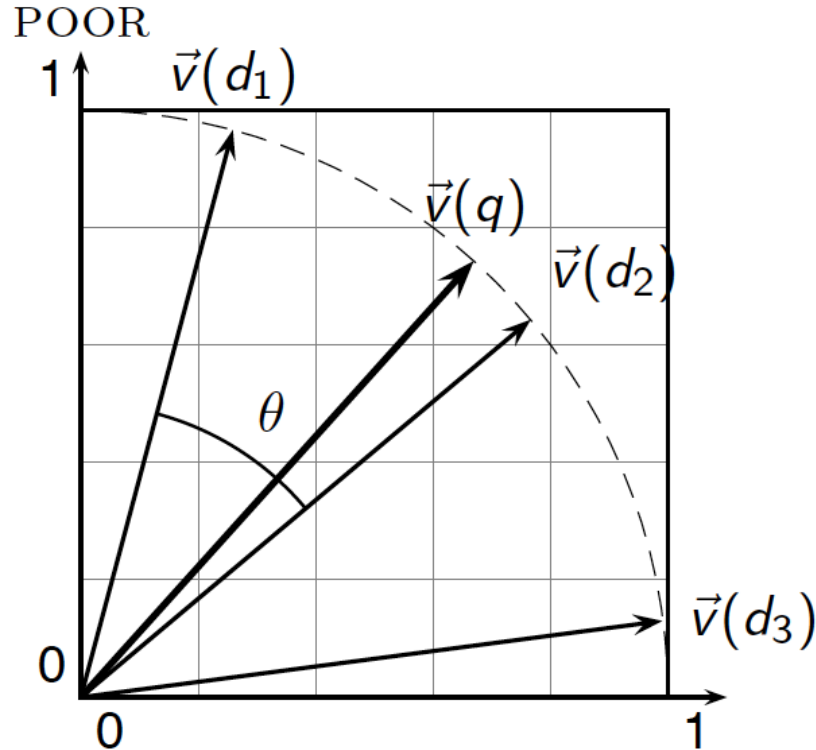
- Dividir un vector por su norma lo convierte en un vector unidad (longitud) (en la superficie de la unidad hiperesférica)

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- El efecto en los dos documentos d y d' (d anexo a sí mismo) de la diapositiva anterior:
 - Tienen vectores idénticos después de la normalización de la longitud.
 - Los documentos largos y cortos ahora tienen pesos comparables.

Ranked Retrieval: $\cosine(query, document)$

- Ilustración de la similitud coseno:



RICH

Ranked Retrieval: similitud coseno entre tres documentos

Que tan similares son las novelas:

SS: *Sentido y Sensibilidad*

OP: *Orgullo y Prejuicio*

CB: *Cumbres Borrascosas*

term	SS	OP	CB
afecto	115	58	20
celoso	10	7	11
chisme	2	0	6
borrascoso	0	0	38

Frecuencia de Terminos (TF)

Nota: para simplificar este ejemplo, no hacemos ponderación idf.

Ranked Retrieval: similitud coseno entre tres documentos

- Log frequency weighting

term	SS	OP	CB
afecto	3.06	2.76	2.30
celosa	2.00	1.85	2.04
chisme	1.30	0	1.78
borrascoso	0	0	2.58

- After length normalization

term	SS	OP	CB
afecto	0.789	0.832	0.524
celosa	0.515	0.555	0.465
chisme	0.335	0	0.405
borrascoso	0	0	0.588

$$\cos(SS, OP) \approx$$

$$0.789 \times 0.832 + 0.515 \times 0.555 + 0.335 \times 0.0 + 0.0 \times 0.0 \approx 0.94$$

$$\cos(SS, CB) \approx 0.79$$

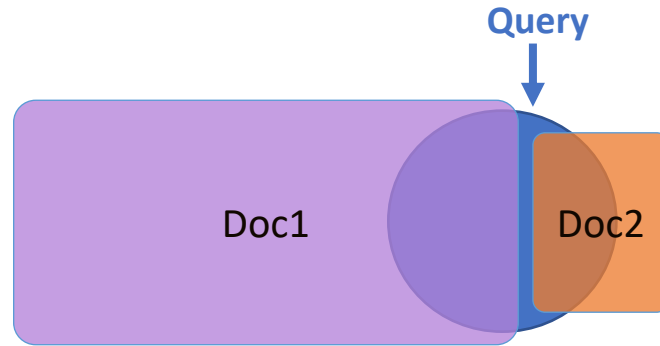
$$\cos(OP, CB) \approx 0.69$$

¿Por qué tenemos $\cos(SS, OP) > \cos(SS, CB)$?

Calculo de la proximidad: Similitud de Coseno

Efecto de la normalización:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$



$$\cos(\text{Query}, \text{Doc2}) > \cos(\text{Query}, \text{Doc1})$$

Ranked Retrieval : ejemplo de tf-idf

Documento: “seguro de carro, seguro de auto”
Query: “el mejor seguro de carro”

Term	Query						Document				Prod
	tf-raw	tf-wt	df	idf	wt	n'lize	tf-raw	tf-wt	wt	n'lize	
auto	0	0	5000	2.3	0	0	1	1	1	0.52	0
mejor	1	1	50000	1.3	1.3	0.34	0	0	0	0	0
carro	1	1	10000	2.0	2.0	0.52	1	1	1	0.52	0.27
seguro	1	1	1000	3.0	3.0	0.78	2	1.3	1.3	0.68	0.53

Que valor tiene N (número de documentos)?

$$\text{Doc length} = \sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$$

$$\text{Score} = 0 + 0 + 0.27 + 0.53 = 0.8$$

Ejemplo de consulta

- La consulta “oro plata camión” en representación vectorial es $\vec{Q} = (0, 0, 0, 0, 0, 0, 1761, 0, 4771, 0, 0, 1761, 0)$
- Las distintas medidas de similitud son:
 - $\text{sim}(\vec{Q}, \vec{W}_1) = 0,00801$
 - $\text{sim}(\vec{Q}, \vec{W}_2) = 0,7561$
 - $\text{sim}(\vec{Q}, \vec{W}_3) = 0,3272$
- Resultados en orden de relevancia: D_2, D_3, D_1

Ranked Retrieval: resumen

- Representar la consulta como un vector de pesos tf-idf.
- Representar cada document como un vector de pesos tf-idf.
- Calcular el score de la similitud coseno para la consulta y cada vector de documento.
- Ranquear los documentos a la consulta con su respectivo score.
- Retornar el Top-K al usuario que hizo la consulta.

Solución en Base de Datos Relacionales

```
Select * from News  
where content ilike '%keyword1%' and content ilike '%keyword2%'
```

En producción la aplicación no sirve ...

¿Por qué?

- ~~1. No se considera la relación léxica de palabras en textos escrito por humanos.~~
- ~~2. Demasiados resultados sin ordenar desesperan al usuario.~~
3. Excesivo tiempo computacional para grandes cantidades de datos.

4.



Modelos de Recuperación de Texto

Índice Invertido

Solución en Base de Datos Relacionales

```
Select * from News where content ilike '%keyword1%'  
and content ilike '%keyword2%'
```

En producción la aplicación no sirve ...

¿Por qué?

3. Excesivo tiempo computacional para grandes cantidades de datos.

- Complejidad Computacional $O(N \cdot n)$
 - N : # tuplas, n : tamaño del texto
- *¿Si aplicamos índice Hash o B+ Tree?*

Matriz Vectorial: Desventajas

Formato disperso:

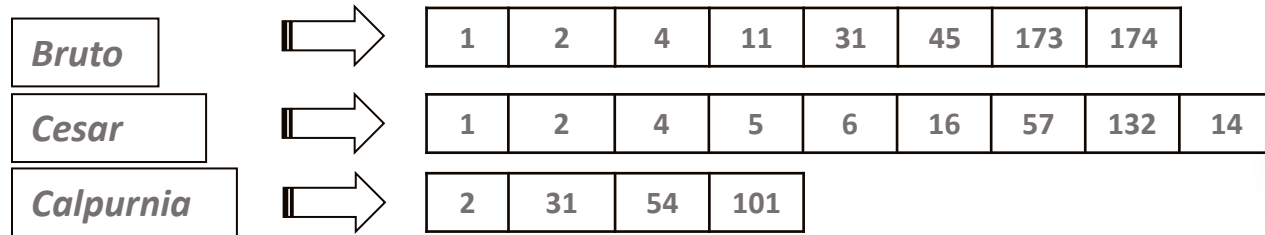
- Asumir:
 - Colección de 1 millón de documentos.
 - Cada documento tiene aproximadamente 1000 palabras.
 - Cada palabra toma 6 bytes, en promedio.
 - De los mil millones de tokens de palabras, 500 000 son únicos (quitando repetidos).
- Entonces:
 - Costo de almacenamiento:
 - $1\text{M} * 1\,000 * 6 = 6\text{GB}$ 
 - Term-Document incidence matrix tomaría:
 - $500,000 * 1,000,000 = 0.5 * 10^{12}$ bits 

Matriz Vectorial: Desventajas

- De las 500 mil millones de entradas, a lo sumo mil millones no son cero.
 - ⇒ Al menos el 99.8% de las entradas son cero.
 - ⇒ Se puede utilizar una **representación dispersa** para reducir el tamaño de almacenamiento!
- Almacenar solo entradas que no sean cero ⇒ **Índice Invertido**.

Índice Invertido

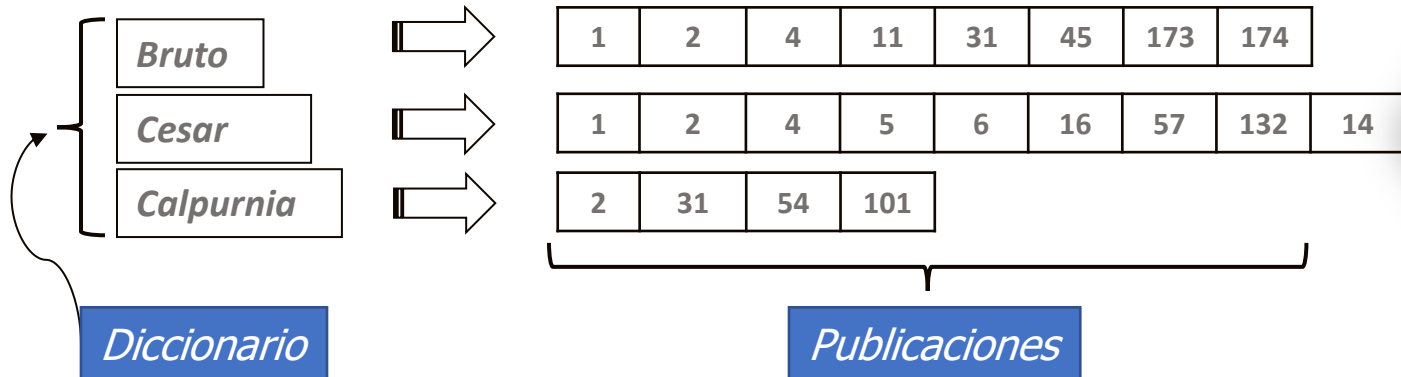
- Para cada termino t , se mapea una lista de todos los documentos que contienen t .
 - Identifique cada documento por un ID de documento, un número de serie del documento.
- ¿Arreglos o Listas?



¿Qué sucede si se agrega la palabra Cesar al documento 14?

Índice Invertido

- Necesitamos listas de publicaciones de **tamaño variable**.
 - En el disco, una lectura continua de publicaciones es normal y mejor.
 - En memoria, se pueden usar listas enlazadas o vectores de longitud dinámica.
 - Algún trade-off entre tamaño / facilidad de inserción.



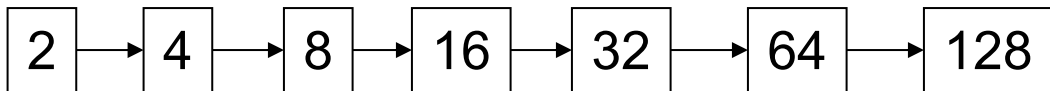
Índice Invertido: consulta AND

- Considere procesar la siguiente consulta:

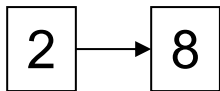
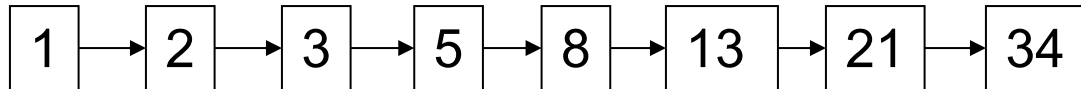
Bruto AND Cesar

- Localizar **Bruto** en el diccionario;
 - Recuperar sus publicaciones.
- Localizar **Cesar** en el diccionario;
 - Recuperar sus publicaciones.
- “**Mezclar**” las dos publicaciones (intersección de conjunto de documentos):

Bruto



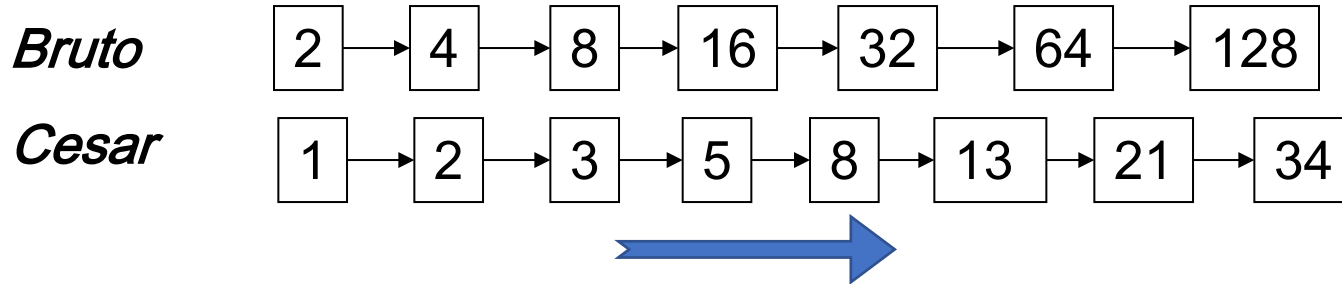
Cesar



Índice Invertido: consulta AND

- **Mezclar:**

- Recorrer las dos publicaciones simultáneamente, en forma lineal sobre el número total de entradas.



Si las longitudes de la lista son n y m , la combinación toma $O(n + m)$ operaciones.

Crucial: las publicaciones deben estar ordenadas por el docID.

Índice Invertido: consulta AND

INTERSECT(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then ADD(answer,  $\text{docID}(p_1)$ )
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

p_1, p_2 – punteros a listas de publicación correspondientes a t_1 y t_2 .
 docID : función que devuelve el Id del documento en la ubicación señalada por p_i .

Índice Invertido: consulta OR

Union

INTERSECT(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(\text{answer}, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then  $p_1 \leftarrow \text{next}(p_1)$ 
9      else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

p_1, p_2 – punteros a listas de publicación correspondientes a t_1 y t_2 .
 docID : función que devuelve el Id del documento en la ubicación señalada por p_i .

$\text{ADD}(\text{answer}, \text{docID}(p_1))$

$\text{ADD}(\text{answer}, \text{docID}(p_2))$

Índice Invertido: consulta NOT

- Ejercicio: adaptar el seucodigo para ejecutar la consulta.

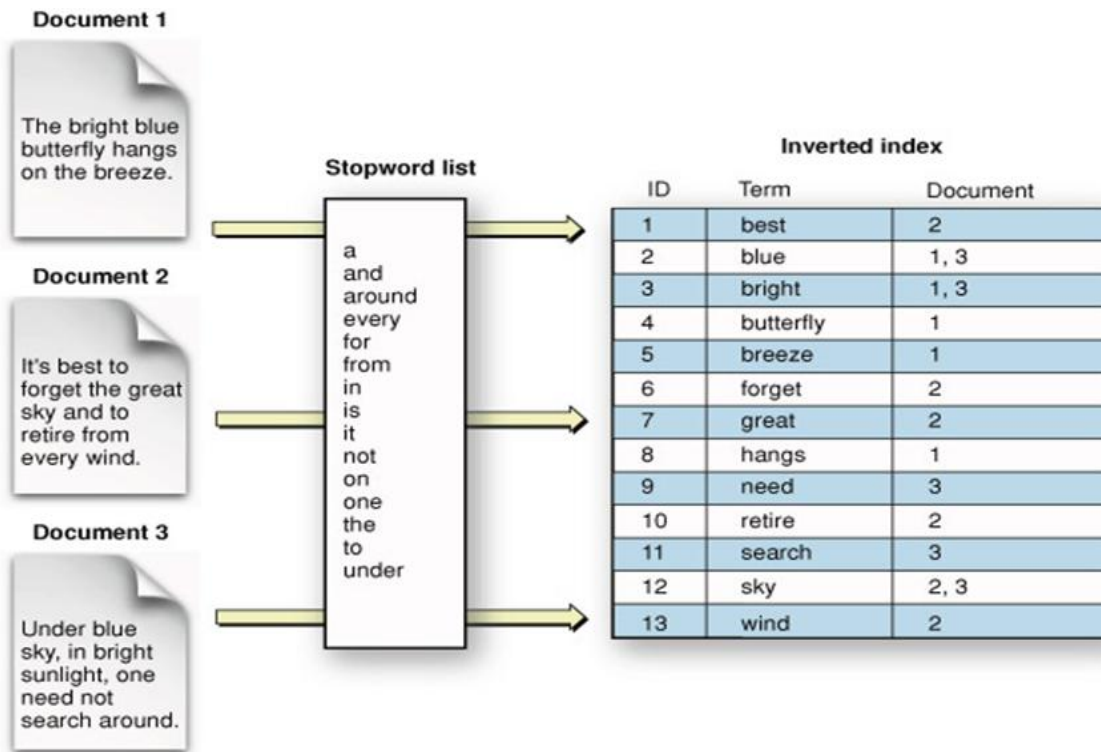
t1 AND NOT t2

ej., Bruto AND NOT Cesar

- ¿Podemos seguir ejecutando el algoritmo en tiempo $O(n + m)$?

- Diseñe el algoritmo.

Índice Invertido: Ejemplo



Índice Invertido: Ejemplo

Query = "blue & sky"



Consulta Booleana usando
operador AND

Complejidad $O(m)$

m : tamaño de la query

Inverted index		
ID	Term	Document
1	best	2
2	blue	1, 3
3	bright	1, 3
4	butterfly	1
5	breeze	1
6	forget	2
7	great	2
8	hangs	1
9	need	3
10	retire	2
11	search	3
12	sky	2, 3
13	wind	2



Blue [1,3]
Sky [2,3]



Result : [3]

Índice Invertido con Similitud de Coseno

Index	
Term	Posting List
casa	[1: 1, 2: 1, 3: 2, 4: 1]
grande	[1: 1, 3: 2]
gato	[2: 1]
bonita	[3: 1]
sol	[1: 2, 4: 2]
brilla	[4: 3]

COSINESCORE(q)

```
1  float Scores[N] = 0
2  float Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top  $K$  components of Scores[]
```



Solución en Base de Datos Relacionales

```
Select * from News where content ilike '%keyword1%'  
and content ilike '%keyword2%'
```

En producción la aplicación no sirve ...

¿Por qué?

- ~~1. No se considera la relación léxica de palabras en textos escrito por humanos.~~
- ~~2. Demasiados resultados sin ordenar desesperan al usuario.~~
- ~~3. Excesivo tiempo computacional para grandes cantidades de datos.~~

Laboratorio 8.2

